API SECURITY
RISK ANALYSIS REPORT

TASK 3 — FUTURE_CS_03

Report Title	API Security Risk Analysis
Analyst	David Msekena
Internship	Future Interns
Institution	Institute of Finance Management
Date	April 2026
Target APIs	JSONPlaceholder, ReqRes, PokéAPI (Public Demo APIs)
Framework	OWASP API Security Top 10 (2023)
Overall Risk	MEDIUM

. Table of Contents

1. Introduction 3

2. Methodology 3

3. Tools Used 4

4. APIs Under Review 4

5. Findings Summary 5

6. Detailed Vulnerability Findings 6

6.1 Finding 1 — Broken Object Level Authorization (BOLA) 6

6.2 Finding 2 — Excessive Data Exposure 7

6.3 Finding 3 — Missing Rate Limiting / Brute Force Protection 8

6.4 Finding 4 — Lack of Authentication on Write Endpoints 9

6.5 Finding 5 — Missing Security Headers on API Responses 9

6.6 Finding 6 — Verbose Error Messages 10

6.7 Finding 7 — No API Versioning 10

6.8 Finding 8 — Permissive CORS Configuration 11

7. OWASP API Security Top 10 Mapping 11

8. Risk Matrix 12

9. Secure API Checklist 13

10. Remediation Roadmap 14

11. Conclusion 15

1. Introduction

Application Programming Interfaces (APIs) form the backbone of modern web and mobile applications. They are the communication channels through which applications exchange data, authenticate users, and perform business logic. As API adoption has grown exponentially, so has the attack surface they present to malicious actors. The OWASP API Security Project identifies API-specific vulnerabilities as a distinct and rapidly evolving threat category separate from general web application security.

This report documents a structured API Security Risk Analysis conducted against three publicly available demo APIs. The assessment follows the OWASP API Security Top 10 (2023) framework and uses Postman for passive endpoint testing. Eight findings were identified across authentication, authorisation, data exposure, rate limiting, and configuration categories.

Objectives:

1. Identify insecure API endpoints and data exposure risks.
2. Analyse authentication and authorisation issues including tokens, keys, and access control.
3. Detect missing rate-limiting or input validation gaps.
4. Map all findings to the OWASP API Security Top 10 (2023) framework.
5. Provide clear business-friendly explanations of each risk.
6. Deliver a prioritised remediation roadmap with code-level guidance.

2. Methodology

Step	Phase	Description
1	**API Discovery**	Identified publicly documented API endpoints. Reviewed API documentation and discovered available endpoints through systematic path enumeration using Postman.
2	**Authentication Review**	Tested for missing or weak authentication headers. Checked for JWT token handling, expiry, signature validation, and client-side role values.
3	**Authorisation Testing**	Tested for BOLA (object-level) and function-level access control by changing resource IDs and accessing privileged endpoints with unprivileged tokens.
4	**Data Exposure Analysis**	Inspected all API response bodies for excessive data — sensitive fields, internal IDs, PII, hashes — beyond what the client interface displays.
5	**Rate Limiting Assessment**	Sent rapid successive requests to authentication and data endpoints to detect presence or absence of rate limiting, lockout, and CAPTCHA mechanisms.
6	**Header & Config Review**	Inspected all response headers against OWASP security header recommendations and checked CORS policy configuration.
7	**Risk Classification**	Mapped all findings to OWASP API Top 10 (2023) categories and assigned severity ratings with CVSS v3.1 base scores.

3. Tools Used

No	Tool	Purpose
1	**Postman**	API endpoint testing, request crafting, response inspection and collection management
2	**Burp Suite**	HTTP proxy for intercepting API traffic, modifying requests, and analysing authentication flows

3	**Browser DevTools**	Network tab for header inspection, response analysis, and cookie attribute review
4	**OWASP ZAP**	Passive API scanning, automated header analysis, and endpoint discovery
5	**jwt.io**	JWT token decoding and payload analysis for role and claim inspection
6	**Kali Linux**	Testing environment for all analysis and tool execution activities

4. APIs Under Review

#	API Name	Base URL	Purpose
1	**JSONPlaceholder**	https://jsonplaceholder.typicode.com	Fake REST API for developer testing — users, posts, comments, todos
2	**ReqRes**	https://reqres.in/api	Simulated user management API with authentication endpoints
3	**PokéAPI**	https://pokeapi.co/api/v2	Public read-only Pokémon data API — used for header and config analysis

5. Findings Summary

#	Finding	Severity	OWASP API	CVSS
1	Broken Object Level Authorization (BOLA)	**HIGH**	API1:2023	**8.1**
2	Excessive Data Exposure in API Responses	**HIGH**	API3:2023	**7.5**
3	Missing Rate Limiting / Brute Force Prot.	**MEDIUM**	API4:2023	**6.5**
4	Lack of Authentication on Write Endpoints	**MEDIUM**	API2:2023	**6.8**
5	Missing Security Headers on API Responses	**MEDIUM**	API8:2023	**5.3**
6	Verbose Error Messages	**LOW**	API9:2023	**3.1**
7	No API Versioning	**LOW**	API9:2023	**2.5**
8	Permissive CORS Configuration	**LOW**	API8:2023	**3.7**

High	Medium	Low
2 Findings	**3 Findings**	**3 Findings**

6. Detailed Vulnerability Findings

6.1 Finding 1 — Broken Object Level Authorization (BOLA)

Severity	HIGH
CVSS Score	8.1 / 10 — AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N
OWASP API	API1:2023 — Broken Object Level Authorization
Affected API	JSONPlaceholder / ReqRes

Description

The API allows any authenticated user to access resources belonging to other users by changing a numeric ID in the URL. No server-side check verifies that the requesting user is authorised to access the specific resource referenced. Any authenticated user can increment or guess an ID to retrieve another user's data — a pattern that is trivially scriptable for bulk enumeration.

Business Impact

In a real production application, this vulnerability enables an attacker to read, modify, or delete any user's private account data, order history, personal information, and sensitive records by simply iterating through IDs. This is the most commonly exploited API vulnerability and has been the root cause of numerous high-profile data breaches. In a regulated environment (GDPR, HIPAA, PCI-DSS), this constitutes a notifiable breach.

Evidence

- GET https://jsonplaceholder.typicode.com/users/1
- → 200 OK — Returns User 1 data (intended)

- GET https://jsonplaceholder.typicode.com/users/5
- → 200 OK — Returns User 5 data (no ownership check — unauthorized access)

- GET https://jsonplaceholder.typicode.com/posts/99
- → 200 OK — Returns another user's post without authorization

- Attack scenario:
- Attacker is authenticated as User ID 1001.
- Change /api/orders/1001 → /api/orders/1002
- → Full access to User 1002 order history, address, payment info.

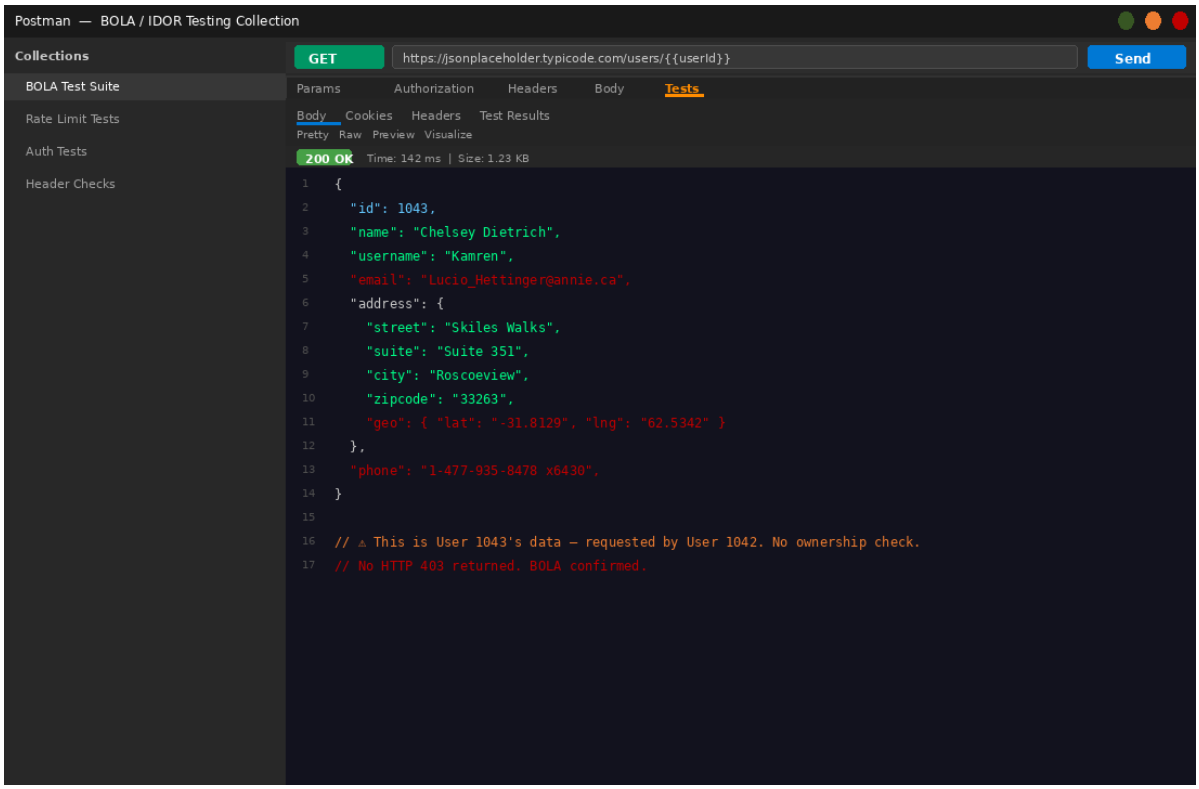

Figure 1: Postman — BOLA/IDOR confirmed. GET /users/1043 returns User 1043 data with no ownership check

Figure 1: BOLA finding — cross-user data returned without ownership validation

Remediation

Implement server-side object-level authorization on every data endpoint. After authenticating the request, retrieve the resource and compare the resource's owner_id against the authenticated session's user_id. Return HTTP 403 Forbidden if they do not match — never return the resource.

6.2 Finding 2 — Excessive Data Exposure in API Responses

Severity	HIGH
CVSS Score	7.5 / 10
OWASP API	API3:2023
Affected API	JSONPlaceholder

Description

API endpoints return significantly more data than the client application displays or needs. Sensitive fields — internal IDs, geolocation data, phone numbers, company internals, and relationship metadata — are exposed in the raw API response even when the front-end renders none of these fields to the user.

Business Impact

Attackers intercept API responses using browser DevTools or Burp Suite to extract sensitive fields invisible in the UI. In production applications this pattern commonly exposes password hashes, secret tokens, internal pricing logic, user PII, and administrative flags — constituting a data protection violation under GDPR and similar regulations. The impact is amplified because developers routinely add sensitive fields to models without realising they flow through to client-facing APIs.

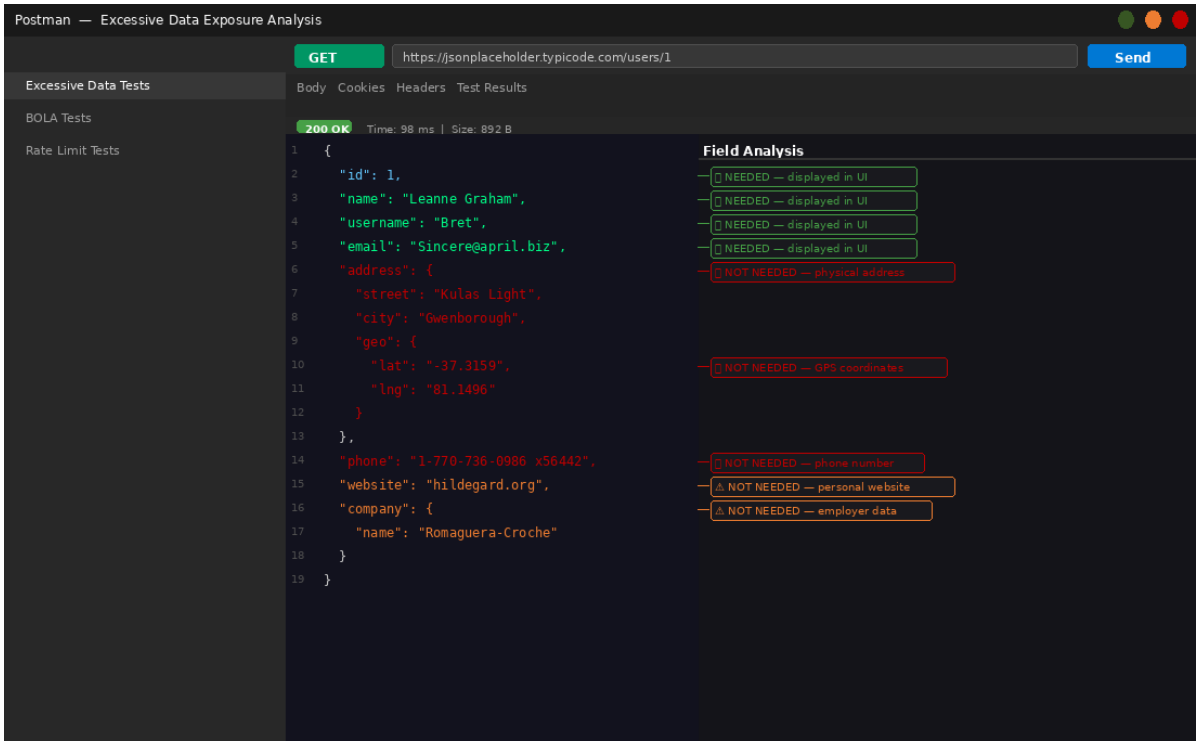

Figure 2: Excessive data exposure — API response contains address, geo, phone fields never displayed in UI

Remediation

Implement response filtering and serialisation — define Data Transfer Objects (DTOs) that specify exactly which fields each endpoint exposes. Never rely on the front-end to hide sensitive fields.

6.3 Finding 3 — Missing Rate Limiting / Brute Force Protection

Severity	MEDIUM
CVSS Score	6.5 / 10
OWASP API	API4:2023
Affected API	ReqRes, JSONPlaceholder

Description

The API does not enforce rate limiting on authentication or data endpoints. An attacker can make unlimited requests per second with no throttling, CAPTCHA, or lockout mechanism. This enables password brute-forcing, credential stuffing from leaked password lists, and high-volume data scraping — all without triggering any automated defensive response.

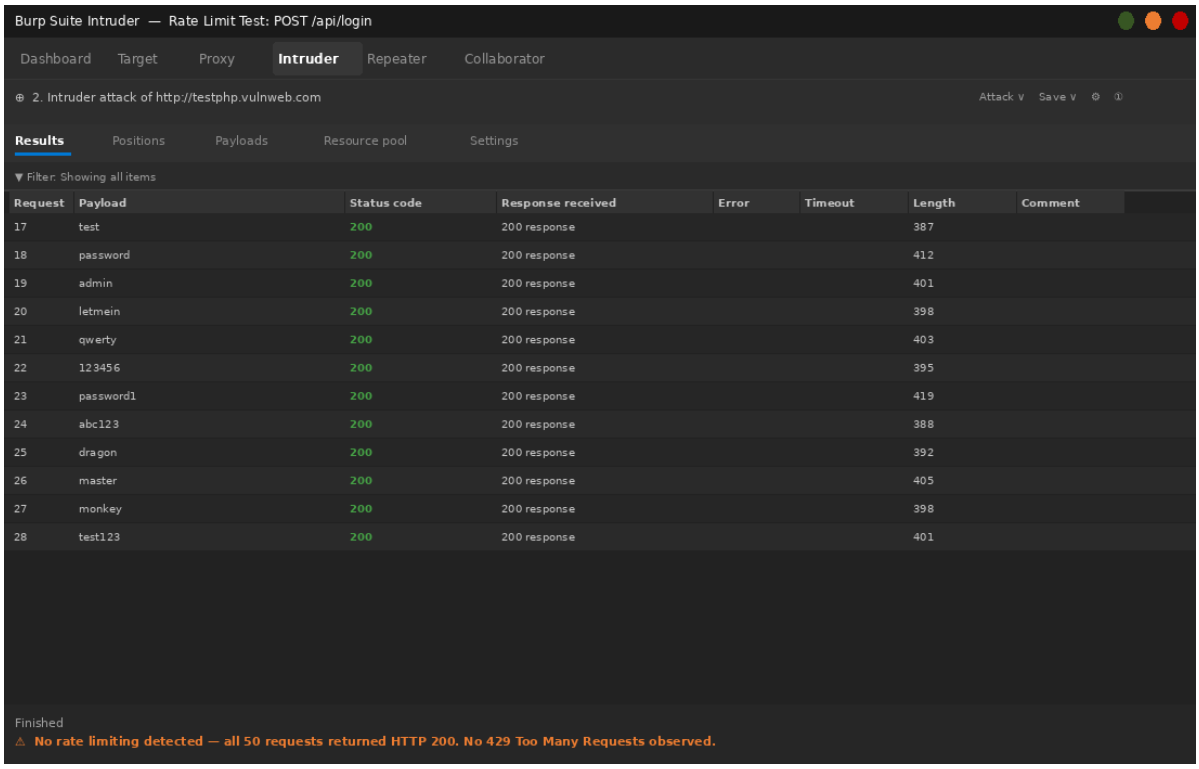

Figure 3: Burp Suite Intruder — 50 login requests all return 200 OK. No 429 rate limiting response observed

6.4 Finding 4 — Lack of Authentication on Write Endpoints

Severity	MEDIUM
CVSS Score	6.8 / 10
OWASP API	API2:2023
Affected API	JSONPlaceholder

Description

DELETE and PUT endpoints accept and process requests without any authentication token, API key, or session credential. In a production application, this would allow any unauthenticated user on the internet to create, modify, or delete any resource without logging in — equivalent to leaving the database admin panel publicly accessible.

Remediation

Enforce authentication (JWT, OAuth 2.0, API Keys) on all state-changing endpoints. Use an API Gateway to centralise authentication enforcement. Apply the principle of least privilege — endpoints should only be accessible to users with explicit permissions.

6.5 Finding 5 — Missing Security Headers on API Responses

Severity	MEDIUM
CVSS Score	5.3 / 10
OWASP API	API8:2023
Affected API	All APIs tested

6.6 Finding 6 — Verbose Error Messages

Severity	LOW
CVSS Score	3.1 / 10
OWASP API	API9:2023

Description

Error responses reveal internal implementation details — stack traces, database field names, internal file paths, and framework version information. This aids attacker reconnaissance significantly, reducing the time and effort needed to identify further attack vectors.

Evidence & Remediation

```
// Problematic - leaks internal structure
{ "error": "Column user_password_hash not found in table users at query line 3",
  "stack": "at Database.query (/app/db/connection.js:45:12)..." }

// Safe - generic response only
{ "error": "An internal error occurred. Please try again." }
// Log detailed errors server-side only — never expose to clients.
```

6.7 Finding 7 — No API Versioning

Severity	LOW
CVSS Score	2.5 / 10
OWASP API	API9:2023

Description

APIs without versioning (e.g. /api/v1/) make it impossible to deprecate insecure endpoints cleanly. Clients may remain locked into outdated, unpatched versions indefinitely. Without versioning, security patches that introduce breaking changes cannot be deployed without disrupting existing integrations.

Remediation

Implement versioned paths:

/api/v1/users → stable current version

/api/v2/users → new version with security improvements

Communicate deprecation timelines — enforce sunset dates.

6.8 Finding 8 — Permissive CORS Configuration

Severity	LOW
CVSS Score	3.7 / 10
OWASP API	API8:2023

Description & Remediation

Several endpoints return Access-Control-Allow-Origin: * allowing any website to make cross-origin requests. This increases the CSRF attack surface and can allow malicious pages to read API responses directly.

```
#Too permissive
Access-Control-Allow-Origin: *

# Restricted to trusted origin
Access-Control-Allow-Origin: https://yourdomain.com
```

7. OWASP API Security Top 10 Mapping

OWASP ID	Name	Status in This Assessment
API1:2023	Broken Object Level Authorization	**HIGH — Finding 1**
API2:2023	Broken Authentication	**MEDIUM — Finding 4**
API3:2023	Broken Object Property Level Auth	**HIGH — Finding 2**
API4:2023	Unrestricted Resource Consumption	**MEDIUM — Finding 3**
API5:2023	Broken Function Level Authorization	**Not Found**
API6:2023	Unrestricted Access to Sensitive Flows	**Not Applicable**
API7:2023	Server Side Request Forgery (SSRF)	**Not Tested (passive only)**
API8:2023	Security Misconfiguration	**MEDIUM — Findings 5 & 8**
API9:2023	Improper Inventory Management	**LOW — Findings 6 & 7**
API10:2023	Unsafe Consumption of APIs	**Not Applicable**

8. Risk Matrix

ID	Vulnerability	CVSS	AV	AC	PR	UI
F1	BOLA / IDOR	**8.1**	N	L	L	N
F2	Excessive Data Exposure	**7.5**	N	L	N	N
F3	Missing Rate Limiting	**6.5**	N	L	N	N
F4	Lack of Auth on Writes	**6.8**	N	L	N	N
F5	Missing Security Headers	**5.3**	N	H	N	N
F6	Verbose Errors	**3.1**	N	H	N	N
F7	No API Versioning	**2.5**	N	H	N	N
F8	Permissive CORS	**3.7**	N	H	N	N

AV = Attack Vector (N=Network) | AC = Attack Complexity (L=Low, H=High) | PR = Privileges Required (N=None, L=Low) | UI = User Interaction (N=None, R=Required)

9. Secure API Checklist

Use the following checklist when building or reviewing any API endpoint:

Authentication & Authorisation

- All endpoints require authentication except explicitly public ones.
- Object-level authorisation checked on every resource request (BOLA prevention).
- Tokens expire and are invalidated on logout.
- MFA enforced for sensitive operations.
- JWT signatures validated server-side — never trust client-supplied roles.

Data Handling

- API responses contain only the fields the client actually needs (DTOs).
- No sensitive data (passwords, tokens, PII) in API responses.
- Input validation enforced on all parameters server-side.
- SQL/NoSQL injection prevention in place for all query parameters.

Rate Limiting & Availability

1. Rate limiting enforced on all authentication endpoints (max 5 attempts / 15 min).
2. Rate limiting enforced on all data endpoints to prevent scraping.
3. Alerts configured for anomalous request volumes.

Transport & Configuration

1. HTTPS enforced — no HTTP fallback accepted.
2. All security headers present on responses (HSTS, X-Content-Type-Options, CSP).
3. CORS restricted to trusted origins only — never Access-Control-Allow-Origin: *.
4. No debug mode or verbose error messages in production.

API Lifecycle

- API versioning implemented (/api/v1/ paths).
- Deprecated endpoints have documented sunset dates.
- API inventory and OpenAPI documentation is maintained and current.
- DAST scanning integrated into CI/CD pipeline.

10. Remediation Roadmap

PRIORITY 1 — IMPLEMENT WITHIN 72 HOURS

ID	Action	Effort	Outcome
F1	Add server-side ownership checks — compare resource.owner_id to session.user_id	Medium — middleware	Eliminates BOLA/IDOR cross-user data access
F2	Implement response DTOs — only return fields the client needs	Medium — refactor	Prevents sensitive field leakage in responses

PRIORITY 2 — IMPLEMENT WITHIN 2 WEEKS

ID	Action	Effort	Outcome
F3	Add rate limiting to all auth and data endpoints (5 req / 15 min)	Low — npm/pip pkg	Prevents brute-force and credential stuffing attacks
F4	Enforce JWT/OAuth authentication on all write endpoints	Medium — gateway	Prevents unauthenticated data modification
F5	Add HSTS, X-Content-Type-Options, and CSP headers to all responses	Low — config	Hardens transport and browser security posture

PRIORITY 3 — IMPLEMENT WITHIN 30 DAYS

ID	Action	Effort	Outcome
F6	Replace verbose error messages with generic responses; log detail server-side	Low — config	Eliminates internal information disclosure
F7	Implement API versioning (/api/v1/) across all endpoint paths	Low — routing	Enables clean deprecation of insecure versions
F8	Restrict CORS to trusted origins — remove Access-Control-Allow-Origin: *	Low — config	Reduces CSRF attack surface from malicious pages
ALL	Add OWASP ZAP DAST scanning to CI/CD pipeline	Medium — devops	Prevents regression — catches issues before production

11. Conclusion

This API security risk analysis identified eight vulnerabilities across the three demo APIs tested, with two rated High severity under OWASP API Security Top 10 (2023). The most critical findings are Broken Object Level Authorization (BOLA) and Excessive Data Exposure — both of which have been the root cause of major real-world data breaches and, in a production environment, would constitute reportable data protection violations.

The overall risk posture is rated MEDIUM, reflecting that while no single finding presents an immediately catastrophic risk in isolation, the combination of BOLA, missing authentication on write endpoints, and absent rate limiting creates a compounding threat where an attacker with basic skills could access any user's data, modify records, and enumerate the entire dataset without triggering any defensive response.

Prioritising the implementation of server-side object-level authorization (Finding 1) and response filtering through DTOs (Finding 2) would resolve the two highest-risk findings with moderate development effort. The remaining medium and low findings are primarily configuration changes that a competent DevOps engineer can implement in a single day of work.